

LABORATORIO 4

Efectos Secundarios, Persistencia y Custom Hooks

OBJETIVOS DEL LABORATORIO

Al finalizar este laboratorio, el estudiante habrá:

- Implementado efectos secundarios con **useEffect** controlando sus dependencias.
 - Aplicado funciones de limpieza para evitar fugas de memoria en temporizadores.
 - Sincronizado datos con **localStorage** mediante **useEffect**.
 - Creado custom hooks reutilizables que encapsulen lógica de estado y efectos.
-

INDICACIONES GENERALES

Desarrolla todos los ejercicios dentro de tu proyecto React con Vite. Crea cada componente y hook en su archivo correspondiente dentro de **src/components/** y **src/hooks/**. Para visualizar el resultado de cada ejercicio, impórtalo provisionalmente en **App.jsx**, aunque el ejercicio final deberá integrarlos todos de forma organizada.

Utiliza únicamente React (**useState**, **useEffect**) y la API de **localStorage** del navegador. No utilices librerías externas para manejo de estado ni notificaciones.

EJERCICIO 1 — SINCRONIZACIÓN CON EL DOM DEL NAVEGADOR

Crea el archivo **src/components/VisorDocumento.jsx**.

El componente debe gestionar un estado numérico contador iniciado en 0. Utiliza **useEffect** para que el título de la pestaña del navegador (**document.title**) muestre siempre el valor actual del contenedor en formato: "Contador: X - Mi App", donde X es el valor actual. El efecto debe ejecutarse cada vez que cambie el contador.

Incluye una función de limpieza que restaure el título a "Mi App" cuando el componente se desmonte.

Renderiza dos botones: uno para incrementar y otro para decrementar el contador, y un botón adicional para "Simular desmontaje" que oculte el componente (mediante un estado booleano en el padre) para verificar que la limpieza funciona correctamente.

EJERCICIO 2 — TEMPORIZADOR CON CONTROL DE INTERVALOS

Crea el archivo **src/components/TemporizadorPomodoro.jsx**.

Implementa un temporizador de cuenta regresiva que inicie en 25 minutos (1500 segundos). Debe mostrar el tiempo restante en formato MM:SS.

Incluye botones para: Iniciar, Pausar y Reiniciar.

Utiliza **useEffect** para gestionar el intervalo (**setInterval**) que decremente el tiempo cada segundo. Recuerda que el intervalo debe limpiarse correctamente cuando el componente se desmonte, cuando se pause el temporizador o cuando se reinicie.

Cuando el tiempo llegue a cero, debe mostrarse una alerta (puede ser un **alert()** nativo o un mensaje visual en el componente) y el temporizador debe detenerse automáticamente.

Valida que el tiempo nunca sea negativo.

EJERCICIO 3 — PERSISTENCIA MANUAL CON LOCALSTORAGE

Crea el archivo **src/components/ConfiguracionUsuario.jsx**.

El componente debe gestionar un formulario con tres campos: nombre (texto), tema (select con opciones 'claro' y 'oscuro') y notificaciones (checkbox). El estado inicial debe intentar leer desde **localStorage** bajo la clave 'config-usuario'. Si no existe, usa valores por defecto: nombre vacío, tema 'claro', notificaciones true.

Implementa la lógica de persistencia manualmente: utiliza **useEffect** para guardar en **localStorage** cada vez que cualquiera de los tres campos cambie. El objeto guardado debe ser un JSON con las tres propiedades.

Muestra debajo del formulario una vista previa en tiempo real de cómo quedaría la configuración guardada. Incluye un botón "Restablecer valores" que limpie el **localStorage** y vuelva al estado inicial.

Maneja posibles errores de **JSON.parse** envolviendo la lectura inicial en un bloque try-catch.

EJERCICIO 4 — EXTRACCIÓN DE LÓGICA EN CUSTOM HOOKS

Crea el archivo **src/hooks/useLocalStorage.js**.

Extrae la lógica del Ejercicio 3 en un hook reutilizable. El hook debe recibir dos parámetros: clave (string) y valorInicial (cualquier tipo). Debe retornar un array con el valor actual y una función setter, exactamente como **useState**.

El hook debe:

- Leer el valor inicial de **localStorage** de forma lazy (solo al montar).
- Sincronizar automáticamente el estado con **localStorage** cuando cambie.
- Manejar errores silenciosamente si **localStorage** no está disponible o los datos están corruptos.

Crea el archivo **src/hooks/useNotificacion.js**.

Implementa un hook que gestione una cola de notificaciones temporales. Debe recibir un parámetro duracion (por defecto 3000ms) y retornar un objeto con:

- **notificacion**: la notificación actual (objeto con id, mensaje y tipo) o null.
- **mostrar(mensaje, tipo)**: función para agregar una notificación.
- **cerrar()**: función para ocultar la notificación manualmente.

Utiliza **useEffect** para que cada notificación desaparezca automáticamente después de la duración especificada. La función de limpieza debe cancelar el timeout si el componente se desmonte o si se muestra una nueva notificación antes de que expire la anterior.

ENTREGA

El estudiante debe entregar un documento con la dirección del repositorio en github con los ejercicios trabajados

Programación Web Avanzada